TSS: Temporal Significance Scheduling for MX-Quantized LLM FFN Inference on CIM

Abstract

Compute-reduction methods for quantized Large Language Model (LLM) inference on Compute-in-Memory (CIM) architectures typically rely on fixed local masks, which preserve hardware regularity but yield a coarse quality-compute tradeoff. Conversely, attempts to improve model quality via flexible sparsity introduce spatial irregularity that severely conflicts with CIM's rigid execution constraints. To overcome this limitation, we propose **Temporal Significance Scheduling (TSS)**, a novel approach that utilizes an alternative significance signal to natively support regular hardware mapping. Rather than masking raw inputs spatially, we reinterpret the block-scale and element-level exponent metadata of the Microscaling (MX) representation as arrival-time signals within a Most Significant Digit (MSD)-first online-arithmetic stream. By temporally bounding operation horizons based on data significance, our method dynamically suppresses ineffectual computations without disrupting spatial regularity. We implement this scheduling via a two-plane Feed-Forward Network (FFN) micro-tile, mapping operations onto channel-parallel, block-serial online-arithmetic engines managed by a lightweight, narrow-integer control plane. Evaluations on Qwen3-0.6B with MXFP8 quantization demonstrate that TSS achieves a superior quality-compute tradeoff, maintaining near-dense baseline perplexity while executing only 50% of the workload. Because block-serial latency is dictated by live-interval contraction rather than raw operation count, this compute reduction translates to a 22–25% layer-wise FFN latency improvement, incurring minimal hardware overheads of only 0.3% in area and ∼2.5% in power. To ensure reproducibility, our transaction-level simulator and hardware evaluation framework are open-sourced anonymously.

Keywords

LLM Inference, Compute-in-Memory, MX Quantization, Temporal Scheduling

ACM Reference Format:

. 2026. TSS: Temporal Significance Scheduling for MX-Quantized LLM FFN Inference on CIM. In . ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Inference in large language models (LLMs) is computationally dominated by feed-forward network (FFN) blocks [11, 16]. Compute-in-memory (CIM) [5, 26] architectures offer a natural execution substrate for accelerating quantized FFN projections by co-locating logic and storage. However, they impose a strict regularity constraint: execution schedules must map to fixed array structures with minimal runtime control divergence [8, 12, 14].

Existing CIM-friendly compute-reduction methods typically enforce rigid local masks—such as structured 2:4 sparsity patterns or static block pruning [20, 27]—to preserve this regularity. While the resulting hardware is straightforward, the tradeoff between model quality and computational reduction is inherently coarse; at moderate sparsity ratios, perplexity degrades disproportionately relative to computational savings [4, 6]. The conventional remedy is to employ more flexible or data-dependent sparsity [8, 12]. However, such approaches introduce additional metadata overhead, runtime irregularity, or require model retraining, thereby eroding the fundamental efficiency advantages of CIM [14].

To overcome these limitations, this paper introduces **Temporal Significance Scheduling (TSS)**. Rather than increasing the complexity of spatial sparsity masks, we reframe the execution problem around *significance-driven temporal truncation*. By refining the significance signal used to determine workload execution, our approach preserves a regular execution substrate while enabling the early termination of less critical operations.

Key observation. Microscaling (MX) quantization [15], increasingly adopted for low-precision LLM deployment [13, 23], stores per-block scale exponents alongside quantized operands. These exponent fields natively encode the relative numerical significance of each block and, at a finer granularity, each element. When computation is structured as most-significant-digit (MSD)-first online arithmetic [7, 22, 28]—where the most-significant digit of each partial product is emitted first—this metadata dictates *when* each contribution accumulates in the output stream. Contributions with large combined exponents arrive early, whereas those with small exponents arrive late, contributing proportionally less to the final result under a fixed execution budget.

Numerically, this behavior aligns directly with significance aware computation and early termination [9, 10]. Ordinarily, temporal truncation targets raw input operands [9] or generic variable precision datapaths [10], which are often hostile to CIM regularity. In our setting, however, MX block- and element-level exponents first induce a temporal alignment across contributions. The hardware scheduler then retains only the most significant prefix of this aligned contribution stream. Consequently, our method acts as a hardware-friendly scheduling policy rather than a standalone truncation heuristic, enabling significance-ordered early termination on regular CIM arrays.

As illustrated in Figure 1, our approach contrasts sharply with spatial sparsity techniques; rather than applying repeated local masks to raw inputs or global keep-sets to spatial entries, it operates

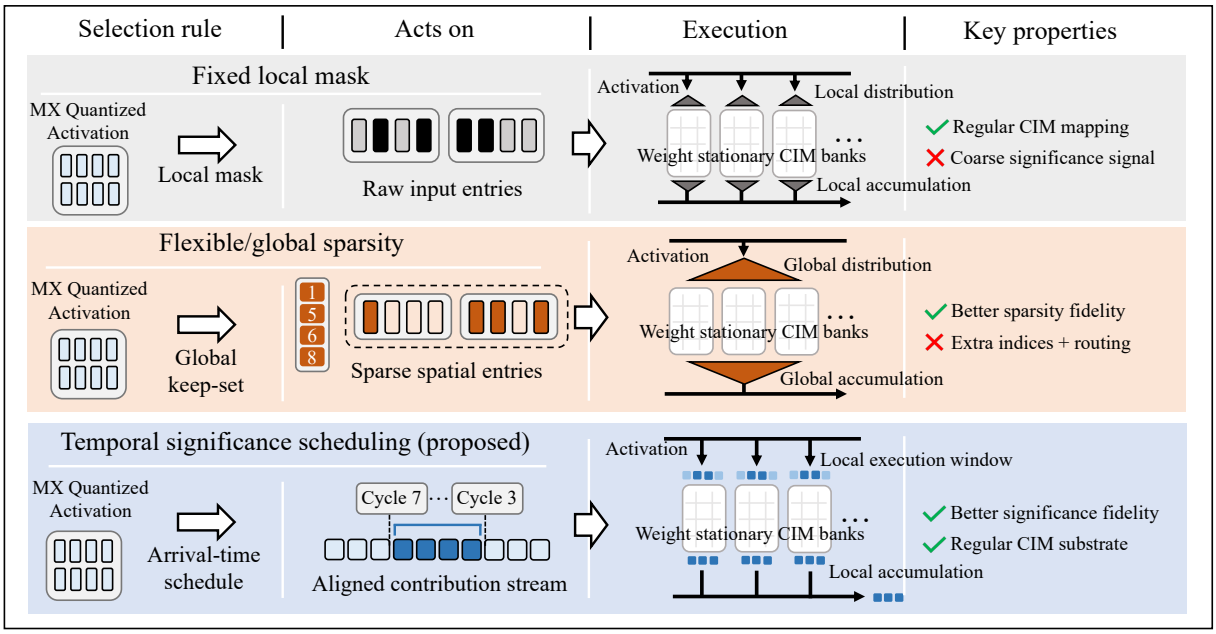


Figure 1: Temporal significance scheduling: an improved significance signal for MX-quantized CIM FFN execution.

by dynamically scheduling arrival times on aligned contribution streams.

Temporal Significance Scheduling (TSS). Operationally, we formulate this abstraction to act directly on an aligned contribution stream. For every block–element pair, the hardware synthesizes an arrival index from existing MX metadata: the activation block-scale exponent, the weight block-scale exponent, and a per-element activation fine-delay code. After temporally aligning the streams, the scheduler applies a calibrated per-channel execution *horizon* to dictate the execution depth. By restricting the digits emitted during MSD-first online multiplication, this mechanism enforces significance-ordered early termination while strictly preserving the underlying regular datapath. This single windowed schedule natively subsumes whole-block skipping, element suppression, and shortened block service intervals under a unified formulation.

Contributions. This paper makes the following contributions:

(1) **A novel execution abstraction.** We introduce TSS, maximizing efficiency by applying significance-aware temporal truncation to aligned contribution streams using existing MX quantization metadata.
(2) **A regular hardware realization.** We propose a metadata-first, two-plane FFN micro-tile tailored for channel-parallel, block-serial online arithmetic. The control plane relies on lightweight integer logic to process metadata, ensuring the data plane remains a highly regular serial–parallel array.
(3) **An improved quality-efficiency tradeoff.** We demonstrate that TSS systematically outperforms activation-gated CIM baselines. Through layer-wise budget redistribution, our approach yields substantial reductions in FFN layer latency and executed operations while preserving baseline model perplexity.
(4) **Open-source simulation framework.** To facilitate future research, we open-source our bit-exact transaction-level model (TLM) and evaluation scripts.

2 Background

This section introduces the inference target, quantization context, and hardware-visible metadata leveraged by temporal significance scheduling.

2.1 GLU-FFN Target

Modern large language models typically employ gated linear unit (GLU) feed-forward networks (FFNs) [17]:

$$\mathbf{y} = W_{\text{down}}\Big(\phi\big(W_{\text{gate}}\,\mathbf{x}\big) \odot W_{\text{up}}\,\mathbf{x}\Big), \tag{1}$$

where ϕ is the gating nonlinearity and $\odot$ denotes element-wise multiplication. The three dense projections (W_{gate}, W_{up}, W_{down}) dominate inference costs, comprising roughly two-thirds of the model's parameters and total operations.

This GLU structure is highly amenable to quantized compute-in-memory (CIM) execution. The large matrix–vector products exhibit regular, token-independent access patterns, while the point-wise gating operation can be evaluated locally between projection stages.

2.2 MX Quantization and Metadata

Microscaling (MX) quantization partitions tensors into fixed-size blocks (typically K=32) [15], assigning a shared block-scale exponent to each. Within a block, individual elements are represented in low-precision floating-point formats. This two-level structure balances dynamic range and storage costs, aligning with emerging industry specifications [3, 13, 23].

TSS relies on four hardware-visible metadata quantities: (1) **Activation block-scale exponent** $\beta_x[n, b]$ (*Native*): The standard, shared E8M0 exponent of activation block b for token n. (2) **Weight block-scale exponent** $\beta_w[p, c, b]$ (*Native*): The shared E8M0 exponent of weight block b on path $p \in \{g, u\}$ for output channel c. (3) **Activation fine-delay code** $\lambda_x[n, b, k]$ (*Derived*): A per-element

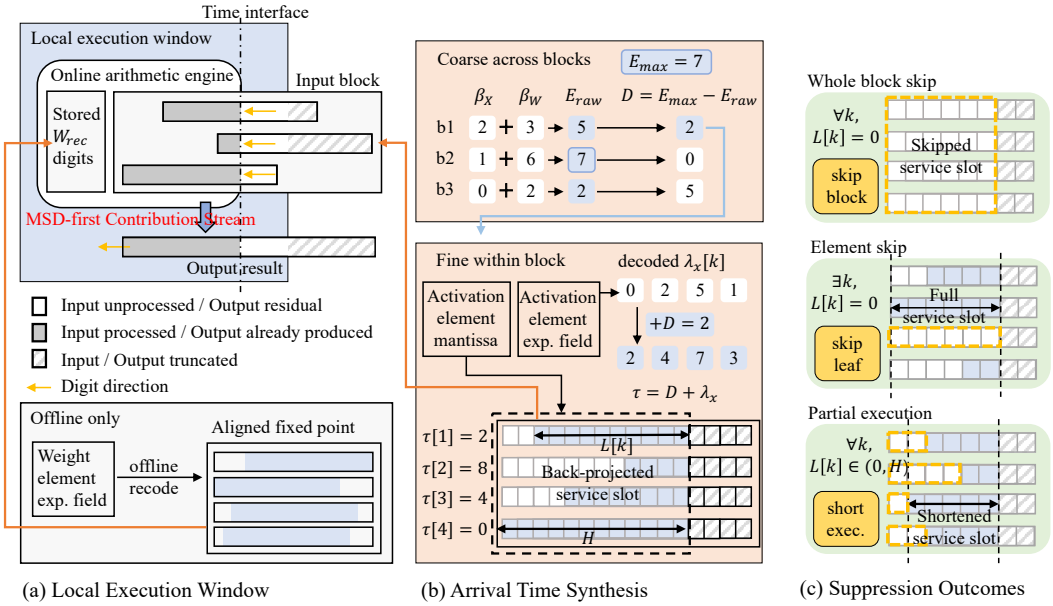


Figure 2: Local execution windows on aligned MSD-first contribution streams.

modifier, extracted directly from the native MXFP element exponent to refine intra-block significance ordering. (4) **Calibrated horizon** $H[p, c]$ (*Introduced*): An offline-calibrated per-path and per-channel scalar that explicitly bounds the execution budget.

2.3 Online Arithmetic Context

CIM arrays designed for low-precision quantized inference can be structured to emit partial products in a most-significant-digit-first (MSD-first) order using online arithmetic [7, 22, 28]. Under this regime, each multiplier leaf generates output digits sequentially from the most significant position, while a reduction tree accumulates these streams across the block.

TSS exploits a key property of this temporal data flow: the *numerical significance of a digit is monotonically non-increasing* relative to its emission order. Consequently, earlier digits contribute exponentially more to the accumulated value than later ones.

When operand blocks possess different combined exponents, their contribution streams must be *aligned* prior to accumulation—mirroring conventional floating-point addition, but distributed temporally. A block with a larger combined exponent begins contributing at an earlier time step, effectively delaying the processing of blocks with smaller exponents. Through this temporal alignment, standard exponent metadata fundamentally acts as an arrival-time signal for the hardware.

3 Temporal Significance Scheduling

This section introduces the core abstractions of Temporal Significance Scheduling (TSS). For notational clarity, the token, projection path (e.g., gate or up), and target channel indices are omitted throughout this discussion. The runtime control logic evaluates exactly four hardware-visible quantities across blocks b and intra-block elements k: • $\beta_x[b]$: activation block-scale exponent, • $\beta_w[b]$: weight block-scale exponent, • $\lambda_x[b, k]$: activation-side fine-delay code, • H : calibrated per-channel horizon.

These variables are narrow integer fields that are either natively present within or efficiently derivable from the MX representation. Because the weight-element exponent is folded offline into the stored, recoded weights, it requires no runtime resolution. Consequently, runtime scheduling relies **strictly on metadata**: it entails only field readouts, narrow integer addition and subtraction, maximum reductions, and comparisons, entirely eliminating the requirement for a floating-point datapath.

3.1 Online Arrival-Time Synthesis

Block-scale metadata establish a coarse relative timing across blocks. As illustrated in Figure 2b, we initially compute a raw exponent, $E[b]$, for each block and determine the per-channel maximum across all blocks, $E_{\max}$:

$$E[b] = \beta_x[b] + \beta_w[b] \quad \text{and} \quad E_{\max} = \max_b E[b]. \tag{2}$$

The arrival time $\tau[b, k]$ for each element is synthesized by refining the coarse block delay $(E_{\max} - E[b])$ to an element granularity using the fine-delay code $\lambda_x[b, k]$:

$$\tau[b, k] = (E_{\max} - E[b]) + \lambda_x[b, k]. \tag{3}$$

Execution is strictly bounded by the local computation window, defined as $W[b, k] = [\,\tau[b, k],\ H\,)$. The active lifespan of a given element corresponds to the duration of this window: $L[b, k] = \max(0, H - \tau[b, k])$. As noted, because the runtime multipliers consume offline-aligned weights, online weight-exponent resolution is completely bypassed during arrival-time synthesis.

3.2 Local Execution Windows on Aligned Contribution Streams

Let $\pi[b,k][m]$ denote the m-th digit produced by the online, Most Significant Digit (MSD)-first multiplication of the activation stream and the offline-aligned weight digits for element k of block b.

As depicted in Figure 2a, TSS evaluates only the segment of the emitted stream that falls within the local execution window. Following alignment by the arrival index $\tau[b,k]$, the executed contribution stream at time step t is:

$$s_{\text{exec}}[b,k] = \begin{cases} \pi[b,k]\big[t-\tau[b,k]\big], & \text{if } t \in W[b,k], \\ 0, & \text{otherwise.} \end{cases} \tag{4}$$

This formulation highlights a primary methodological distinction from traditional masking-based approaches: rather than imposing a static spatial mask on raw input operands, the scheduler restricts the *aligned contribution stream* generated by MSD-first multiplication. Consequently, the executed entity is a temporal window applied to a significance-ordered stream, avoiding the superimposition of sparsity patterns on the original tensor data.

3.3 Unified Compute-Suppression View

As illustrated in Figure 2c, this window-based formulation naturally unifies three execution outcomes without requiring divergent control paths. When $L[b,k] = 0$ for all elements within a block or for a specific element k, the architecture executes a **whole-block skip** or an **element skip**, respectively, effectively suppressing the corresponding leaf multipliers along with their associated memory accesses and switching activity. Conversely, when $0 < L[b,k] < H$, the system executes a **partial window**, gracefully scaling down computation such that the leaf multiplier processes strictly fewer digits than a dense baseline.

3.4 Offline Horizon Calibration

The horizon H dictates the execution length of the aligned contribution stream. To control the compute budget, we instantiate horizons via a three-stage offline calibration process. First, a **uniform horizon** establishes anchor operating points by applying a single global integer H across all paths and channels. Second, **channel-wise SNR redistribution** adjusts per-channel horizons to satisfy a target signal-to-noise ratio, providing an effective initialization for fine-grained tuning.

Finally, **layer-wise L_2 redistribution** dynamically reallocates per-layer horizons to populate the inter-integer gaps. Formally, this stage solves a constrained optimization problem: minimizing the L_2 norm of the output divergence between the quantized and dense layers ($\min \|Y_{\text{quant}} - Y_{\text{dense}}\|_2$) subject to a global computational budget constraint. This refinement preserves the uniform horizon's quality frontier while yielding a continuous, dense set of practical operating points. Ultimately, horizon calibration acts strictly as a budget-control mechanism; the core efficiency gain is provided by the alignment-aware temporal scheduling itself.

4 Hardware Realization

Figure 3 illustrates the proposed **metadata-first two-plane FFN micro-tile**, which realizes the scheduling strategy from Section 3 with minimal hardware overhead. The tile design employs a weight-stationary Digital Compute-In-Memory (CIM) architecture: weight SRAM is locally integrated within every projection lane, with only the activation contribution streams traveling through the compute array.

4.1 Tile-Level FFN Mapping

The proposed micro-tile maps the GLU-FFN onto three execution stages guided by a *channel-parallel, block-serial* execution strategy (Figure 3a):

(1) **Front-end projections.** The W_{gate} and W_{up} computations execute concurrently within channel-parallel projection lanes, while the B input blocks assigned to each channel are serialized in time.
(2) **Local gating.** The SiLU nonlinearity and element-wise product ($\phi(\text{gate}) \odot \text{up}$) are evaluated directly on the intermediate results using a reused serial–parallel multiplier.
(3) **Output projection.** The gated output is forwarded to S local W_{down} shards to compute the final projection.

4.2 Metadata-First Control Plane

By operating predominantly on metadata, the control plane enforces TSS with a minimal area footprint.

Scale prepass. A metadata-only scan computes the cross-block maximum exponent $E_{\max}$ and resolves coarse block delays. This prepass strictly accesses narrow metadata arrays (β_x and β_w) and low-bitwidth adders to update local registers, entirely bypassing the activation streams and Digital-CIM weight arrays.

Shared λ_x decode. The activation fine-delay vector $\lambda_x[n,b,0:31]$ is decoded once in a shared central cluster and broadcast to the relevant projection lanes.

Per-lane window builders. Each lane combines the shared fine-delay vector with its local coarse delay and channel horizon to parameterize the per-element window configurations (τ and L, as defined in Section 3). These window limits are double-buffered locally, allowing the control logic to continuously prepare block $b+1$ while block b executes.

4.3 Online-Arithmetic Data Plane

Exploiting the weight-stationary CIM substrate, each projection lane houses two block-processing engines corresponding to the front-end `gate_proj` and `up_proj`. Each engine comprises $K = 32$ serial–parallel (SP) multiplier leaves, a five-level block-local overlap-add (OLA) reduction tree, and a channel accumulator that merges successive serialized blocks:

$$\text{acc}[p,b+1,c] = \text{OLA}\big(\text{acc}[p,b,c],\ s_{\text{blk}}[p,b,c]\big).$$

After both accumulators finish, standard fixed-point logic applies the SiLU activation and evaluates $\text{gated}_c = \text{SiLU}(\text{gate}_c) \odot \text{up}_c$. This intermediate result is passed to the local `down_proj` shards, which contain dedicated SP multiplier banks and row accumulators. By passing windowed data streams directly alongside sideband metadata, this interface seamlessly connects the stages without requiring wide, full-precision buses.

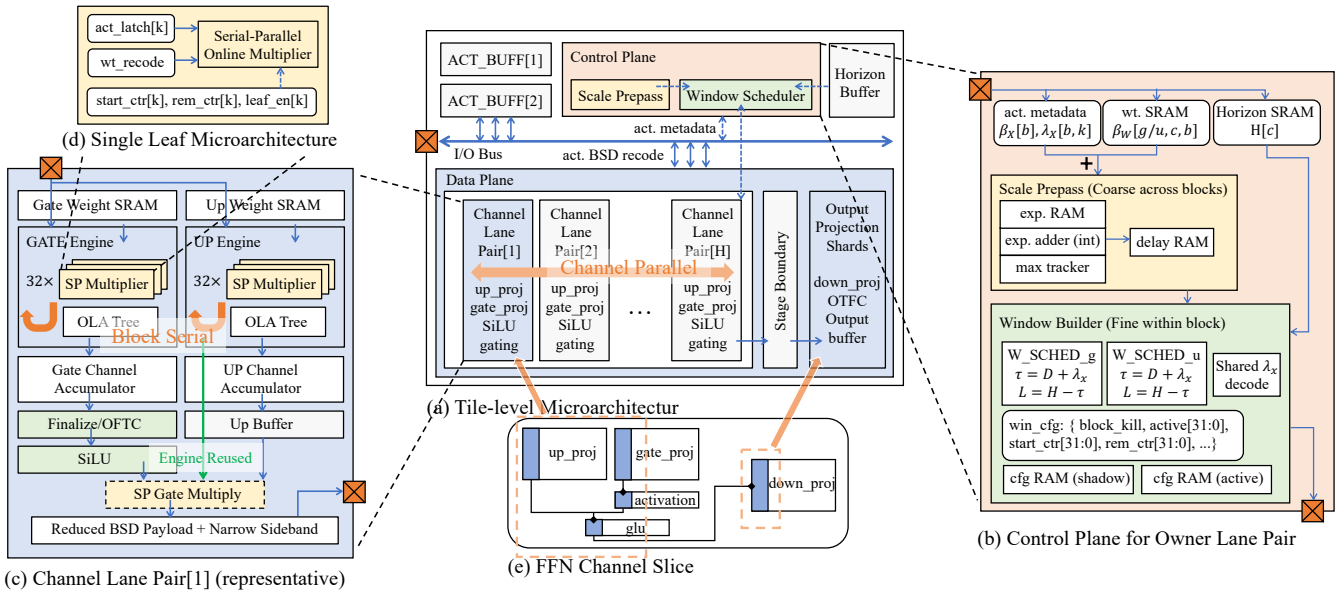


Figure 3: Channel-parallel, block-serial two-plane FFN micro-tile.

4.4 Hardware Savings via Operation Gating

The three scheduling outcomes derived in Section 3 directly translate into dynamic hardware savings: • **Whole-block skips** entirely bypass the local arithmetic trees and corresponding memory reads, proportionally contracting the elapsed block execution span. • For **element skips** and **partial-window execution**, idle SP multiplier leaves and their local memory interfaces are clock-gated. Furthermore, inactive OLA-tree ancestors are dynamically gated once their residual states drain, consistently reducing switching activity even if the maximal block baseline interval remains unchanged.

5 Experimental Methodology

This section details the simulation infrastructure, target models, hardware measurement methodology, and evaluation protocols that underpin the empirical results presented in Section 6.

Transaction-Level Simulation. We developed a custom open-source Python-based transaction-level model (TLM) [1, 2] to simulate the precise MSD-first data flow of the proposed micro-tile (Section 4). The TLM tracks all serial-parallel multiplications, block-local overlap-add reductions, and channel accumulations at digit-level granularity. It reproduces the aligned contribution streams (Eq. 4) and windowed execution on a cycle-by-cycle basis. To ensure bit-exact numerical outputs with respect to the hardware scheduling abstraction, the TLM strictly avoids the use of floating-point surrogates within the inner loop.

Model Setup and Calibration. We evaluate our architecture using the Qwen3-0.6B model [25] from the HuggingFace Transformers library [24]. To isolate the architectural effects of the feed-forward network (FFN), all experiments target the GLU-style FFN sublayers while keeping the attention sublayers at the dense baseline.

Both weights and activations are quantized to the MXFP8 format (8-bit MX microscaling with E8M0 block-scale exponents and a block size of $K = 32$). Block-scale exponents, activation fine-delay codes, and weight element exponents are extracted directly from this MX representation. An offline recoding process stores the aligned weight mantissa in a fixed-point format. Horizon calibration leverages a held-out set of 1024 tokens randomly sampled from the WikiText-2 training split, whereas all model quality metrics are evaluated on the disjoint WikiText-2 test split.

Hardware Measurement Flow. Our hardware measurement flow combines direct circuit implementation with macro-level modeling at the 28nm technology node. To evaluate area and power accurately, we partition the hardware into three distinct components: the control plane, the compute-in-memory (CIM) data plane, and SRAM storage.

The *control plane* encompasses the complete TSS runtime scheduling logic and Horizon SRAM. We implement the control plane in Cadence Virtuoso and validate each component with transistor-level simulation in a commercial 28 nm process. This includes the metadata scale prepass (computing $E[b]$ and $E_{\max}$ from β_x and β_w), the shared λ_x decode/broadcast logic, per-lane window builders (forming τ and L for horizon comparison), and all associated local registers and control logic.

The CIM data plane—comprising the serial-parallel multipliers [21], overlap-add reduction trees, and channel accumulators—is synthesized using the Synopsys Design Compiler at a 28 nm technology node. Finally, weight storage is synthesized as SRAM macros using an ARM memory compiler for the same 28 nm node. The total reported per-channel area and power represent the direct sum of these three partitions.

Baselines and Comparison Protocol. All evaluated methods, including baselines, are normalized to a matched executed-digit ratio, defined as $\rho_{\text{exec}} = N_{\text{exec}}/N_{\text{exec,dense}}$. We compare our proposed architecture against three baseline methodologies: (1) **Dense**

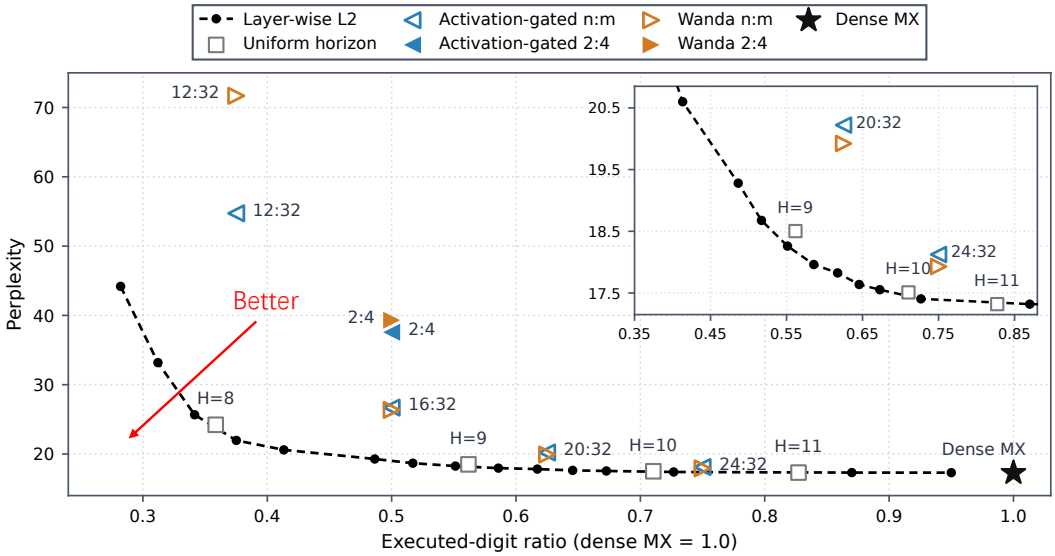


Figure 4: Quality-compute Pareto frontier for MX-quantized Qwen3-0.6B FFN inference.

MXFP8 Baseline: Full MXFP8 execution with no compute suppression, serving as the primary dense anchor. (2) **Dynamic Activation-Gated CIM**: Methods that apply fixed, local masks derived dynamically from runtime activation statistics. ([8, 12]). (3) **Static Calibrated Pruning**: A calibration-based static pruning baseline (Wanda [18]) without retraining. This serves as the primary comparative baseline for our calibrated layer-wise L_2 redistribution variant.

We map the resulting $n:m$ sparsity structure directly to the equivalent executed-digit ratio by $\rho_{\text{exec}} = n/m$. For the proposed architecture, we evaluate three operating configurations: *uniform horizon* (a baseline staircase anchor), *channel-wise SNR-based redistribution* (an ablation variant), and *layer-wise L_2 redistribution* (which establishes our main Pareto frontier).

6 Evaluation Results

Our evaluation isolates the numerical fidelity of temporal significance scheduling before demonstrating its system-level benefits. We establish the quality-compute Pareto frontier, quantify the architectural benefits of reducing executed digits, validate the offline-online partitioning, and report the hardware efficiency.

6.1 Core Quality-compute Pareto Frontier

Figure 4 presents the quality-compute trade-off across all baseline configurations.

Pareto efficiency of TSS. Across all executed-digit ratios, layer-wise L_2 redistribution consistently achieves the lowest perplexity, converging to the dense MX baseline as the ratio approaches 1.0. The discrete operating points of the uniform horizon policy fall along this same curve, indicating that quality improvements stem primarily from the TSS abstraction itself—specifically, executing local windows on aligned contribution streams. The redistribution mechanism interpolates between integer-horizon steps to provide fine-grained budget control, rather than fundamentally shifting the Pareto frontier.

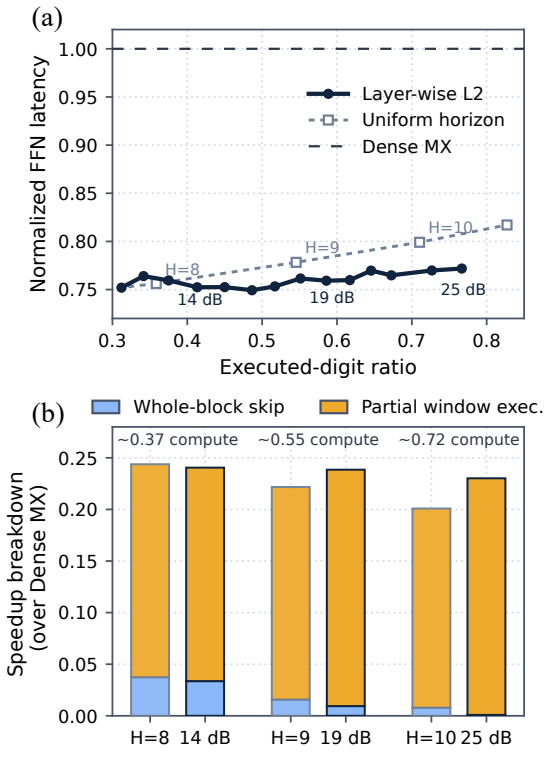


Figure 5: (a) Normalized FFN latency versus executed-digit ratio. (b) Acceleration contribution breakdown.

Comparison with structured sparsity. The widely adopted 2:4 structured sparsity pattern yields substantially higher perplexity for both the online activation-gated mask and the Wanda, reflecting the limitations of coarse, fixed local patterns at this quantization level. Relaxing the constraint to an n:32 pattern improves quality by expanding the selection scope, but layer-wise L_2 redistribution consistently outperforms it at equivalent compute budgets. This performance gap widens at low executed-digit ratios, where preserving the most significant contributions under a tight budget is especially important.

6.2 Layer-Wise Latency under Block-Serial Execution

Figure 5 translates the projected compute reduction into realized FFN latency under a channel-parallel, block-serial execution model.

Stable layer-level acceleration. As shown in Figure 5(a), layer-wise L_2 redistribution reduces normalized FFN latency by 22–25% across the practical operating range. At higher executed-digit ratios, where the uniform horizon approach yields diminishing returns, layer-wise redistribution maintains a distinct advantage via fine-grained reallocation: channels contributing heavily to output quality retain higher precision, while less significant channels receive shorter service windows.

Acceleration breakdown. Figure 5(b) breaks down this latency reduction. Whole-block skipping unconditionally eliminates block slots, providing a bounded acceleration that peaks at approximately 20% of the total gain. The majority of savings stem from partial-window execution contracting the maximal live interval within partially executed blocks; this effect dominates at higher executed-digit ratios. As average horizons widen, fewer blocks qualify for full skips, yet partial-window contraction remains highly effective because individual contribution streams often initiate late relative to the channel horizon.

These results validate the architectural claims from Section 3: while whole-block skipping reduces block slots, partial-window execution reduces overall executed digits. In practice, these dual mechanisms yield substantial layer-level latency reductions as live intervals contract concurrently across multiple channels.

6.3 Validation of the Offline-Online Partitioning

Our design statically stores calibrated horizons and dynamically synthesizes local execution windows at runtime. Figure 6 validates three key properties of this offline-online split. First, regarding **budget controllability** (Figure 6a), layer-wise L_2 redistribution provides near-linear, fine-grained tuning of the executed-digit ratio across the practical 0.3–0.7 regime, unlike the coarse step-function of a uniform horizon. Second, **calibration** is highly robust (Figure 6b); model quality stabilizes with only ~1k calibration tokens—fewer than Wanda's requirement of 256 sequence samples, each containing 2k tokens [18]—because the lightweight procedure merely redistributes pre-existing budgets based on layer-level statistics. Finally, the **metadata-source ablation** (Figure 6c) confirms that dynamic activation statistics are critical. Weight-only redistribution severely degrades perplexity ($> 10^3$), whereas combining global activation and weight metadata (*layer-wise* redistribution) achieves the lowest perplexity, outperforming independent *channel-wise* optimization by ~1.6 points.

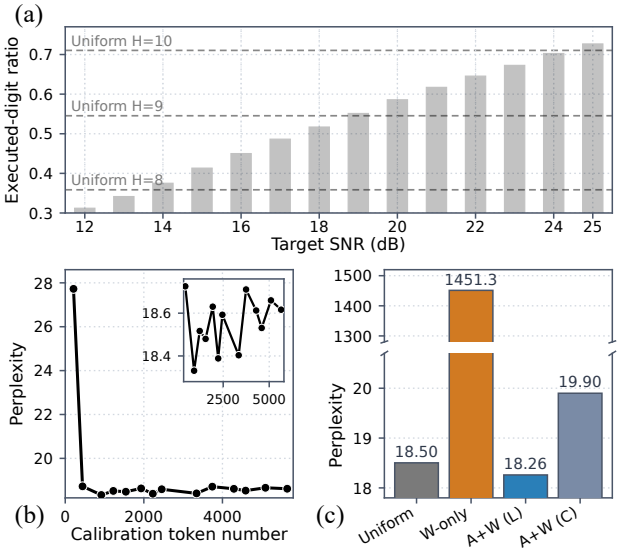


Figure 6: Validation of the offline-online split. (a) Executed-digit ratio versus initialization target SNR for layer-wise L_2 redistribution and uniform horizon. (b) Perplexity versus calibration sample size. (c) Metadata-source ablation.

6.4 Hardware Cost

Figure 7 details the per-channel area and power breakdown of the synthesized hardware design across four functional domains.

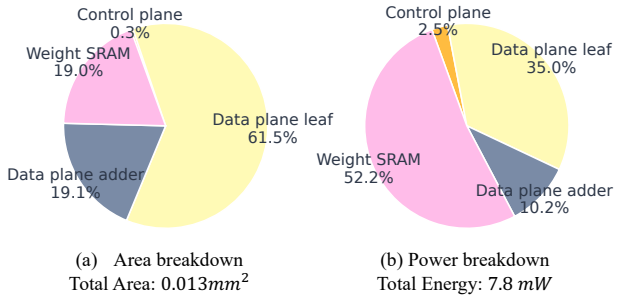


Figure 7: Per-channel area and power breakdown.

Negligible Control-Plane Overhead. The control plane consumes just 45.1 µm^2 (<0.35% of the 13 008 µm^2 total per-channel area) and 0.198 mW (≈ 2.5% of the 7.81 mW power budget). This validates our core architectural claim: replacing per-element spatial masks with per-channel horizons and block-level control reduces routing complexity from an $O(n)$ multiplexing network to lightweight scalar logic. Consequently, the control overhead is lower than that of representative spatial-sparsity accelerators.

Data-Plane and SRAM Costs. The online arithmetic leaf dominates the area footprint (7 996.6 μm^2, 61.5%) due to the digit-serial multiply–accumulate datapath required for MX-format computation. The adder tree accounts for 19.1% of area and 10.3% of power. The weight SRAM occupies 19.0% of area and dictates overall power consumption (4.07 mW, 52.2%), reflecting the memory-intensive profile typical of compute-in-memory (CIM) macro designs. Crucially, the proposed scheduling policy strictly confines overhead to the control plane, leaving the adder tree and SRAM costs uninflated.

Table 1: Comparison with relative hardware overheads of sparsity-aware accelerators. Control overhead denotes the area fraction attributed to sparsity and scheduling logic. Prior values are quoted as reported in their respective publications. For TSS, the value is calculated as the synthesized control-plane area divided by the total per-channel tile area.

Architecture	Policy	Precision	Control Overhead
Z.Yue [27]	Activation 2:4	FP16	26.0%
VEGETA [8]	Flexible *n*:*m*	BF16	15.4%
FlexCIM [12]	Flexible *n*:*m*	INT8	5.9%
SDP [19]	Activation 1:2	INT16	3.5%
TSS (ours)	**TSS**	**MXFP8**	**0.35%**

Comparison with Prior Accelerators. Table 1 contextualizes the TSS control overhead against recent sparsity-aware CIM and near-memory accelerators implemented in comparable (≤28 nm) nodes. By dividing the synthesized control-plane area by the total per-channel tile area (45.1 μm^2/13,008 μm^2), TSS demonstrates a control overhead of just 0.35%. As shown, this result suggests materially lower control-area cost than spatial sparsity support logic.

7 Conclusion

Existing spatial sparsity methods inherently clash with the strict structural regularity demanded by compute-in-memory (CIM) architectures. To resolve this tension, we introduce *Temporal Significance Scheduling (TSS)*, an execution paradigm that shifts significance-aware compute reduction from the spatial domain to the temporal domain. By repurposing native Microscaling (MX) block- and element-scale exponents as arrival-time signals within a most-significant-digit (MSD)-first arithmetic stream, our approach dynamically truncates low-significance operations without imposing divergent control flows on the underlying hardware array.

Empirical and physical synthesis results validate the architectural viability of this temporal abstraction. Implemented via a metadata-first, two-plane micro-tile, the required scheduling logic is entirely decoupled from the high-precision datapath, dictating a negligible overhead of 0.3% in area and 2.5% in power. When evaluated on the MXFP8-quantized Qwen3-0.6B model, our design achieves a 22–25% FFN latency reduction. By establishing a superior quality-compute Pareto frontier compared to conventional activation-gated and static weight pruning baselines—all without requiring model retraining—this work demonstrates that the temporal dimension of digit-serial CIM execution provides a low-overhead, natively regular mechanism for dynamic compute suppression in modern LLM accelerators.

References

[1] Fernando Aguirre, Abu Sebastian, and Manuel Le Gallo. 2024. Hardware implementation of memristor-based artificial neural networks. *Nat. Commun.* 15, 1 (2024), 1974.

[2] Lukai Cai and Daniel Gajski. 2003. Transaction level modeling: an overview. In *CODES+ISSS*. 19–24.

[3] Bita Darvish Rouhani, Ritchie Zhao, Venmugil Elango, Rasoul Shafipour, Mathew Hall, Maral Mesmakhosroshahi, Ankit More, Levi Melnick, Maximilian Golub, Girish Varatkar, et al. 2023. With shared microexponents, a little shifting goes a long way. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–13.

[4] Elias Frantar and Dan Alistarh. 2023. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International conference on machine learning*. PMLR, 10323–10337.

[5] Hidehiro Fujiwara, Haruki Mori, Wei-Chang Zhao, Mei-Chen Chuang, Rawan Naous, Chao-Kai Chuang, Takeshi Hashizume, Dar Sun, Chia-Fu Lee, Kerem Akarvardar, et al. 2022. A 5-nm 254-TOPS/W 221-TOPS/mm 2 fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 65. IEEE, 1–3.

[6] Simla Burcu Harma, Ayan Chakraborty, Elizaveta Kostenok, Danila Mishin, Dongho Ha, Babak Falsafi, Martin Jaggi, Ming Liu, Yunho Oh, Suvinay Subramanian, et al. 2024. Effective interplay between sparsity and quantization: From theory to practice. *arXiv preprint arXiv:2405.20935* (2024).

[7] Zhijun Huang and Milos D Ercegovac. 2001. FPGA implementation of pipelined on-line scheme for 3-D vector normalization. In *The 9th Annual IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM'01)*. IEEE, 61–70.

[8] Geonhwa Jeong, Sana Damani, Abhimanyu Rajeshkumar Bambhaniya, Eric Qin, Christopher J Hughes, Sreenivas Subramoney, Hyesoon Kim, and Tushar Krishna. 2023. Vegeta: Vertically-integrated extensions for sparse/dense gemm tile acceleration on cpus. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 259–272.

[9] Patrick Judd, Jorge Albericio, Tayler Hetherington, Tor M. Aamodt, and Andreas Moshovos. 2016. Stripes: Bit-serial deep neural network computing. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1–12. doi:10.1109/MICRO.2016.7783722

[10] Win-San Khwa, Ping-Chun Wu, Jui-Jen Wu, Jian-Wei Su, Ho-Yu Chen, Zhao-En Ke, Ting-Chien Chiu, Jun-Ming Hsu, Chiao-Yen Cheng, Yu-Chen Chen, Chung-Chuan Lo, Ren-Shuo Liu, Chih-Cheng Hsieh, Kea-Tiong Tang, and Meng-Fan Chang. 2024. 34.2 A 16nm 96Kb Integer/Floating-Point Dual-Mode-Gain-Cell-Computing-in-Memory Macro Achieving 73.3-163.3TOPS/W and 33.2-91.2TFLOPS/W for AI-Edge Devices. In *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 67. 568–570. doi:10.1109/ISSCC49657.2024.10454447

[11] Yu Jung Lo and Samuel Williams. 2015. Roofline model toolkit: A practical tool for architectural and program analysis. In *HPCS PMBS*. Springer, 129–148.

[12] Asit Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. 2021. Accelerating sparse deep neural networks. *arXiv preprint arXiv:2104.08378* (2021).

[13] Asit Mishra, Dusan Stosic, Simon Layton, and Paulius Micikevicius. 2025. Recipes for Pre-training LLMs with MXFP8. *arXiv preprint arXiv:2506.08027* (2025).

[14] Akshat Ramachandran, Souvik Kundu, Arnab Raha, Shamik Kundu, Deepak K Mathaikutty, and Tushar Krishna. 2025. Accelerating LLM inference with flexible N: M sparsity via a fully digital compute-in-memory accelerator. In *2025 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE, 1–7.

[15] Bita Darvish Rouhani et al. 2023. OCP Microscaling Formats (MX) Specification, Version 1.0.

[16] Siddharth Samsi, Dan Zhao, and Joseph McDonald. 2023. From words to watts: Benchmarking the energy costs of large language model inference. In *IEEE HPEC*. IEEE, 1–9.

[17] Noam Shazeer. 2020. Glu variants improve transformer. *arXiv* (2020).

[18] Mingjie Sun, Zhuang Liu, Anna Bair, and J Zico Kolter. 2023. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695* (2023).

[19] Fengbin Tu, Yiqi Wang, Ling Liang, Yufei Ding, Leibo Liu, Shaojun Wei, Shouyi Yin, and Yuan Xie. 2022. SDP: Co-designing algorithm, dataflow, and architecture for in-SRAM sparse NN acceleration. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 1 (2022), 109–121.

[20] Fengbin Tu, Yiqi Wang, Ling Liang, Yufei Ding, Leibo Liu, Shaojun Wei, Shouyi Yin, and Yuan Xie. 2023. SDP: Co-Designing Algorithm, Dataflow, and Architecture for In-SRAM Sparse NN Acceleration. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 1 (2023), 109–121. doi:10.1109/TCAD.2022.3172600

[21] Muhammad Usman, Miloš D. Ercegovac, and Jeong-A Lee. 2023. Low-latency online multiplier with reduced activities and minimized interconnect for inner product arrays. *Journal of Signal Processing Systems* 95, 7 (2023), 777–796.

[22] Julio Villalba, Tomas Lang, and Javier Hormigo. 2011. Radix-2 multioperand and multiformat streaming online addition. *IEEE Trans. Comput.* 61, 6 (2011), 790–803.

[23] Wenjun Wang, Shuo Cai, Congkai Xie, Mingfa Feng, Yiming Zhang, Zhen Li, Kejing Yang, Ming Li, Jiannong Cao, and Hongxia Yang. 2025. InfiR2: A Comprehensive FP8 Training Recipe for Reasoning-Enhanced Language Models. *arXiv preprint arXiv:2509.22536* (2025).

[24] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. HuggingFace's Transformers: State-of-the-art Natural Language Processing. arXiv:1910.03771 [cs.CL] https://arxiv.org/abs/1910.03771

[25] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[26] Shimeng Yu, Hongwu Jiang, Shanshi Huang, Xiaochen Peng, and Anni Lu. 2021. Compute-in-memory chips for deep learning: Recent trends and prospects. *IEEE circuits and systems magazine* 21, 3 (2021), 31–56.

[27] Zhiheng Yue, Xujiang Xiang, Yang Wang, Ruiqi Guo, Huiming Han, Shaojun Wei, Yang Hu, and Shouyi Yin. 2025. 14.4 A 51.6TFLOPs/W Full-Datapath CIM Macro Approaching Sparsity Bound and <2-30 Loss for Compound AI. In *2025 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 68. 1–3. doi:10.1109/ISSCC49661.2025.10904702

[28] Yiren Zhao, John Wickerson, and George A Constantinides. 2016. An efficient implementation of online arithmetic. In *2016 international Conference on Field-programmable Technology (FPT)*. IEEE, 69–76.